COSC 434 – Advanced Web Programming

Lab Session VI

Protecting the Recipe App Using Middleware

Spring 2026

Dr. Roaa Soloh
CMS Department

Objective

In this lab, you will continue the existing Recipe App and implement middleware to control access to selected routes.

The main goal is to practice the middleware concept from the lecture in a focused and practical way. By the end of the session, your application should allow all users to view recipes, while only a demo logged-in user can access recipe management actions such as create, edit, update, and delete.

Learning Outcomes

By the end of this lab, you should be able to:

- explain the purpose of middleware in Laravel,
- create a custom middleware,
- register a middleware alias,
- apply middleware to a single route,
- apply middleware to a route group,
- test both public and protected routes.

Part 1: Fork and Set Up the Project

Task 1: Fork the Repository

Open the repository on GitHub and fork it to your own account.

Your repository should become:

```
https://github.com/your-username/cosc434-recipe-manager-spring2026
```

Task 2: Clone Your Fork

Clone your own fork locally.

Use the usual Git commands to:

- clone the repository,

- move into the project folder.

Example:

```
git clone https://github.com/your-username/cosc434-
recipe-manager-spring2026.git
cd cosc434-recipe-manager-spring2026
```

Task 3: Prepare the Project

Set up the Laravel project so it runs correctly on your machine.

You are expected to:

- install dependencies,
- create the `.env` file,
- generate the application key,
- configure the database,
- run migrations,
- start the project.

Typical commands include:

```
composer install
cp .env.example .env
php artisan key:generate
php artisan migrate
php artisan serve
```

Checkpoint

Before continuing, open the app in the browser and confirm that the Recipe App is working.

Part 2: Identify the Access Problem

Task 4: Inspect the Current Behavior

Before adding middleware, test the current routes.

Check whether a visitor can access:

- the recipe list,
- a single recipe,
- the create page,
- the edit page,
- update and delete actions.

Think

Based on the lecture, not all these routes should behave the same way.

Write down:

- which routes should remain public,
- which routes should be protected.

You should reach a result similar to this idea:

- viewing routes stay public,
- management routes become protected.

Part 3: Add a Simple Demo Login State

In this lab, you are not required to build full authentication.
Instead, you will simulate login using the session.

Task 5: Add Demo Login and Logout Routes

Open `routes/web.php`.

Add two simple routes:

- one route to mark the user as logged in,
- one route to remove that login state.

Your implementation should use:

- `Request`
- session storage
- redirect after the action

A possible structure is:

```
Route::get('/login-demo', function (Request $request) {
    // store something in session
    // redirect somewhere useful
});

Route::get('/logout-demo', function (Request $request)
{
    // remove the session value
    // redirect somewhere useful
});
```

What You Need to Decide

You need to decide:

- what session key to use,
- what value represents the login state,
- where the user should be redirected.

Hint

Keep this part very simple.
A single session flag is enough.

Example idea:

- a key such as `logged_in`
- a true/false style value

Checkpoint

After implementing this part:

- visiting `/login-demo` should create the login session state,

- visiting `/logout-demo` should remove it.

Part 4: Create the Middleware

Task 6: Generate the Middleware

Create a middleware named:

```
EnsureUserIsLoggedIn
```

Use the Artisan command for middleware generation.

Task 7: Complete the Middleware Logic

Open the generated file inside:

```
app/Http/Middleware
```

You will find the `handle()` method.
Your job is to complete its logic.

The method should:

- check whether the session indicates a logged-in state,
- redirect the user if access is denied,
- allow the request to continue if access is allowed.

A simplified structure is:

```
public function handle(Request $request, Closure
$next): Response
{
    if (/* condition for not logged in */) {
        return redirect(/* where? */)
            ->with(/* optional flash message */);
    }

    return $next($request);
}
```

What This Part Does

Inside this method:

- `$request` gives you access to the current request and the session,
- the `if` condition decides whether access is allowed,
- `return $next($request);` passes the request forward,
- redirecting stops the request before it reaches the controller.

Think

Before coding, answer these:

- What exactly should your condition check?
- Where should the user go if access is denied?
- Why must `$next($request)` be returned?

Part 5: Register the Middleware Alias

Task 8: Register the Middleware

Once the middleware class is ready, register an alias for it in the proper Laravel configuration area.

Choose a short alias that clearly reflects its purpose.

For example, your alias might look conceptually like:

```
'demo.auth' =>
\App\Http\Middleware\EnsureUserIsLoggedIn::class
```

You need to place this in the correct middleware registration section for your Laravel version.

Think

Why is it better to use an alias in routes instead of writing the full class name each time?

Part 6: Protect One Route First

Task 9: Apply Middleware to a Single Route

Do not protect all routes immediately.

First, apply the middleware to only one route:

- the create recipe route

Your route should end up with the idea of:

```
Route::get('/recipes/create', [RecipeController::class,
'create'])
    ->middleware('demo.auth');
```

Checkpoint

Test the following:

1. visit `/logout-demo`,
2. try opening `/recipes/create`,
3. confirm that access is denied,
4. visit `/login-demo`,
5. try `/recipes/create` again,
6. confirm that access is allowed.

Think

Why is it better to test one protected route first before moving to a full route group?

Part 7: Protect a Route Group

Task 10: Group the Protected Routes

Once your single-route test works, protect the rest of the recipe management routes using a middleware group.

The protected actions should include:

- create
- store
- edit
- update
- delete

The public actions should remain:

- index
- show

Your structure should follow the idea below:

```
Route::get('/recipes', [RecipeController::class,
'index'])->name('recipes.index');
Route::get('/recipes/{recipe}',
[RecipeController::class, 'show'])-
>name('recipes.show');

Route::middleware('demo.auth')->group(function () {
    // protected management routes here
});
```

Your Job

Inside the middleware group, place all routes related to modifying recipe data.

Think

Why is grouping better here than attaching the same middleware separately to every route?

Part 8: Make Testing Easier

Task 11: Add Quick Access Links

To make your testing easier, add visible links or buttons in the interface for:

- demo login,
- demo logout.

For example, in a Blade file you may add something like:

```
<a href="/login-demo">Demo Login</a>
<a href="/logout-demo">Demo Logout</a>
```

You may style them as buttons if you want.

Task 12: Show Feedback Messages

Display flash messages so the interface clearly shows:

- login success,
- logout success,
- access denied.

A typical Blade idea is:

```
@if(session('success'))
    <div class="alert alert-success">
        {{ session('success') }}
    </div>
@endif
```

You may add a similar block for error messages.

Checkpoint

After this part, your interface should make it easy to test both states:

- logged in,
- logged out.

Part 9: Optional Extension

Task 13: Hide Management Buttons for Guests

As an optional improvement, hide buttons such as:

- Add Recipe,
- Edit,
- Delete

when the user is not in the demo logged-in state.

The logic may follow this idea:

```
@if(session('logged_in'))
    <!-- show management action -->
@endif
```

Important

This improves the interface only.
It is not the real protection.

The route must still be protected by middleware.

Testing Checklist

Before submitting, make sure all the following work correctly.

Public Access

- You can open the recipe list while logged out.
- You can open a single recipe while logged out.

Protected Access While Logged Out

- You cannot access the create page.
- You cannot access the edit page.
- You cannot perform store, update, or delete actions.

Protected Access While Logged In

- You can activate the demo login.
- You can access the create page.
- You can create a recipe.
- You can edit a recipe.

- You can delete a recipe.
- After logout, protected access is blocked again.

Submission Requirements

What You Must Submit

You must submit:

1. the link to your forked GitHub repository,
2. your completed lab pushed to that repository,
3. a short updated `README.md`.

README Requirements

Your `README.md` should include:

- your full name,
- course and lab title,
- a short paragraph explaining what you implemented,
- confirmation that you tested both:
  - a single protected route,
  - a protected middleware group.

Git Requirement

Commit and push your final work to your fork.

A suitable commit message could be:

```
git add .
git commit -m "Implemented middleware protection for
recipe management routes"
git push origin main
```

Moodle Submission

Submit your forked GitHub repository link on Moodle.